离散数学：算法与复杂度 (Algorithms & Complexity) 复习讲义

整理自：DS-chapter3-QYT-2026 PPT

目录

1 本章概览 **3**

2 算法基础与典型问题 **3**

2.1 从问题到算法 . . . 3
2.2 伪代码与算法描述 . . . 3
2.3 算法的基本性质 . . . 4
2.4 典型算法问题 . . . 4
2.5 线性搜索 . . . 4
2.6 二分搜索 . . . 5
2.7 排序问题 . . . 5
2.8 冒泡排序 . . . 6
2.9 插入排序 . . . 6
2.10 快速排序（自学） . . . 7
2.11 字符串匹配（自学） . . . 7
2.12 贪心算法 . . . 8
2.13 优化问题与贪心思想 . . . 8
2.14 找零问题 . . . 8
2.15 美国硬币系统的最优性 . . . 8
2.16 面值系统会影响贪心正确性 . . . 9
2.17 贪心调度 . . . 9
2.18 停机问题（拓展） . . . 10

3 函数增长与渐近效率 **10**

3.1 选择算法时考虑的因素 . . . 10
3.2 算法分析的目标 . . . 11
3.3 渐近效率 . . . 11
3.4 Big-O 记号 . . . 11
3.5 定义与直观含义 . . . 11

3.6 Big-O 的注意事项 . . . 11
3.7 Big-O 例题 . . . 12
3.8 多项式的 Big-O . . . 12
3.9 常见估计 . . . 12
3.10 常见复杂度函数 . . . 13
3.11 函数组合的增长 . . . 13
3.12 Big-Omega 与 Big-Theta . . . 14
3.13 Big-Omega . . . 14
3.14 Big-Theta . . . 14

4 算法复杂度 **15**
4.1 复杂度的含义 . . . 15
4.2 三类分析 . . . 15
4.3 基本语句复杂度规则 . . . 15
4.4 求最大元素的复杂度 . . . 16
4.5 线性搜索复杂度 . . . 16
4.6 二分搜索复杂度 . . . 16
4.7 冒泡排序复杂度 . . . 17
4.8 插入排序复杂度 . . . 17
4.9 矩阵乘法复杂度 . . . 17
4.10 布尔矩阵乘积复杂度 . . . 17
4.11 复杂度类别与 P/NP（拓展） . . . 18
4.12 常见术语 . . . 18
4.13 P 与 NP 问题 . . . 18

5 全章总结 **18**
5.1 课件习题 . . . 19

1 本章概览

第 3 章围绕算法及其效率分析展开。核心问题是：面对一个明确的问题，如何给出能在有限步骤内求解的过程，并用与具体机器无关的方式评价它的运行时间和空间消耗。

本章主线

1. **算法 (Algorithms)**：问题、算法定义、伪代码、算法性质，以及搜索、排序、贪心算法等例子。
2. **函数增长 (Growth of Functions)**：渐近效率、Big-O、Big-Omega、Big-Theta 及常见复杂度函数。
3. **算法复杂度 (Complexity of Algorithms)**：最坏、最好、平均情形分析，以及典型算法的复杂度计算。

2 算法基础与典型问题

2.1 从问题到算法

许多领域中的关键问题都可以被抽象为：给定满足条件的输入，要求输出具有特定性质的结果。解决这类一般问题通常分为两步：

1. **精确定义问题**：明确输入结构、合法输入范围和期望输出。
2. **给出求解过程**：写出一个有限步骤的过程，使它对每个合法输入都能产生期望输出。

算法 (algorithm) 是执行计算或解决问题的一组有限、精确的指令。

例：求有限序列中的最大元素

```
procedure max(a1, a2, ..., an: integers)
    max := a1
    for j := 2 to n
        if max < aj then max := aj
    return max
{max is the largest element}
```

该算法把当前已知最大值保存在变量 `max` 中，并依次与后续元素比较。循环结束后，`max` 就是整个序列的最大元素。

2.2 伪代码与算法描述

算法可以用自然语言描述，也可以用伪代码描述。**伪代码 (pseudocode)** 介于自然语言和具体编程语言之间，常使用 C++、Java 等语言中常见的顺序、分支、循环结构。

- 伪代码不依赖某一种编程语言，便于表达算法思想。
- 程序员可以据此实现具体程序。
- 伪代码有助于分析算法运行时间，避免被某种语言或硬件细节干扰。

2.3 算法的基本性质

性质	含义
输入 (Input)	算法从指定集合中取得输入值。
输出 (Output)	算法根据输入产生指定集合中的输出，输出即问题的解。
正确性 (Correctness)	对每一组合法输入，算法都应产生正确输出。
有限性 (Finiteness)	对任何输入，算法都必须在有限步后给出输出。
有效性 (Effectiveness)	每一步都必须能够被正确执行，并且在有限时间内完成。
一般性 (Generality)	算法应适用于该类问题的所有合法实例，而不是只解决某个特殊例子。

2.4 典型算法问题

课件中列出四类常见问题：

1. **搜索 (Searching)**：在列表中找到某个特定元素的位置。
2. **排序 (Sorting)**：把列表元素按递增顺序排列。
3. **字符串匹配 (String Matching)**：在文本串中找到模式串出现的位置。
4. **优化问题 (Optimization Problems)**：在所有可能输入或方案中，使某个量达到最大值或最小值。

2.5 线性搜索

线性搜索 (linear search) 从列表开头开始逐个检查，直到找到目标元素或扫描完整个列表。

```
procedure linear search(x: integer, a1, a2, ..., an: distinct integers)
    i := 1
    while (i <= n and x != ai)
        i := i + 1
    if i <= n then location := i
    else location := 0
```

```
    return location
{location is the subscript of the term that equals x,
 or 0 if x is not found}
```

线性搜索不要求列表有序；代价是最坏情况下需要检查所有元素。

2.6 二分搜索

二分搜索 (binary search) 要求输入列表已经递增排列。它每次取当前搜索区间的中点，并根据中点元素与目标值的大小关系丢弃一半区间。

```
procedure binary search(x: integer, a1, a2, ..., an: increasing integers)
    i := 1          {left endpoint of search interval}
    j := n          {right endpoint of search interval}
    while i < j
        m := floor((i + j)/2)
        if x > am then i := m + 1
        else j := m
    if x = ai then location := i
    else location := 0
    return location
```

例：在列表中搜索 19

列表为

$$1, 2, 3, 5, 6, 7, 8, 10, 12, 13, 15, 16, 18, 19, 20, 22.$$

第一次取第 8 个元素 10，由于 $19 > 10$，搜索区间缩小为第 9 到第 16 个位置；第二次取第 12 个元素 16，继续缩小到第 13 到第 16 个位置；第三次取第 14 个元素 19，再缩小到第 13 到第 14 个位置；第四次取第 13 个元素 18，最终区间只剩第 14 个位置，因此返回 19 的位置。

> 注意
>
> 按课件给出的列表，19 的下标是 14。课件最后一句写作返回位置 16，应为笔误；位置 16 对应的元素是 22。

2.7 排序问题

排序 (sorting) 是把列表元素按递增顺序排列，例如数值顺序、字母顺序等。排序重要的原因包括：

- 大量计算资源用于排序，尤其是数据库、检索和报表类应用。
- 已有许多本质不同的排序算法，它们的优缺点被广泛研究。
- 排序算法非常适合说明计算机科学中的基本概念。

课件中提到的排序算法包括 binary、insertion、bubble、selection、merge、quick、tournament 等。本章重点示例为冒泡排序、插入排序和快速排序。

2.8 冒泡排序

冒泡排序 (bubble sort) 对列表进行多趟扫描。每趟中，若发现相邻两个元素逆序，就交换它们。

```
procedure bubblesort(a1, ..., an: real numbers with n >= 2)
    for i := 1 to n - 1
        for j := 1 to n - i
            if aj > aj+1 then interchange aj and aj+1
{a1, ..., an is now in increasing order}
```

每完成一趟，当前未排序部分中的最大元素就会被放到它最终应在的位置。例如输入序列为

$$3, 2, 4, 1, 5.$$

对应过程如下：

趟数	本趟结束后的序列	说明
1	$2, 3, 1, 4, 5$	最大元素 5 已在正确位置，4 被推到倒数第二个候选位置
2	$2, 1, 3, 4, 5$	第二大元素 4 已在正确位置
3	$1, 2, 3, 4, 5$	前三个元素完成排序
4	$1, 2, 3, 4, 5$	序列保持有序

2.9 插入排序

插入排序 (insertion sort) 从第二个元素开始，逐步把当前元素插入到它在前面已排序部分中的正确位置。

```
procedure insertion sort(a1, ..., an: real numbers with n >= 2)
    for j := 2 to n
        i := 1
        while aj > ai
            i := i + 1
        m := aj
        for k := 0 to j - i - 1
            aj-k := aj-k-1
        ai := m
{a1, ..., an is now in increasing order}
```

对输入 $3, 2, 4, 1, 5$，插入排序过程为：

步骤	序列	说明
i	$2,3,4,1,5$	前两个位置交换
ii	$2,3,4,1,5$	第三个元素仍在正确位置
iii	$1,2,3,4,5$	第四个元素插入开头
iv	$1,2,3,4,5$	第五个元素仍在正确位置

2.10 快速排序（自学）

快速排序 (quick sort) 基于**分治 (divide and conquer)** 思想：选取枢轴 (pivot)，把数组划分为小于和大于枢轴的两部分，再递归排序两部分。

```
void quickSort(int arr[], int left, int right) {
    int i = left, j = right;
    int tmp;
    int pivot = arr[(left + right) / 2];

    while (i <= j) {
        while (arr[i] < pivot) i++;
        while (arr[j] > pivot) j--;
        if (i <= j) {
            tmp = arr[i];
            arr[i] = arr[j];
            arr[j] = tmp;
            i++;
            j--;
        }
    }

    if (left < j) quickSort(arr, left, j);
    if (i < right) quickSort(arr, i, right);
}
```

补充：快速排序平均复杂度为 $\Theta(n\log n)$，最坏情况下为 $\Theta\big(n^2\big)$。实际实现中常通过随机选取枢轴或“三数取中”等策略降低最坏情况出现概率。

2.11 字符串匹配（自学）

字符串匹配问题要求在文本串 T 中找到模式串 P 出现的位置。最直接的朴素算法会枚举每一个可能起点，并逐字符比较，若 $|T|=n$、$|P|=m$，最坏复杂度为 $\Theta(nm)$。更高效的算法包括 KMP、Boyer–Moore 和 Rabin–Karp 等。

2.12 贪心算法

2.13 优化问题与贪心思想

优化问题要求在所有可行方案中最小化或最大化某个参数，例如：

- 在两个城市之间寻找总里程最短的路线；
- 用尽可能少的比特编码消息；
- 用最少光纤连接网络节点；
- 安排尽可能多的互不冲突活动。

贪心算法 (greedy algorithm) 每一步都做当前看起来最好的选择。局部最优选择不一定推出全局最优解；因此设计贪心算法后，必须证明它总能得到最优解，或给出反例说明它并不总是正确。

2.14 找零问题

问题：用美国硬币面值 25、10、5、1 分为 n 分找零，并使硬币总数最少。

贪心策略：每一步选择不超过剩余金额的最大面值硬币。例如 $n = 67$：

$$67 \to 42 \to 17 \to 7 \to 2 \to 1 \to 0,$$

对应硬币为 $25, 25, 10, 5, 1, 1$，共 6 枚。

```
procedure change(c1, c2, ..., cr: coin values with c1 > c2 > ... > cr;
                 n: a positive integer)
    for i := 1 to r
        di := 0
        while n >= ci
            di := di + 1
            n := n - ci
    return d1, d2, ..., dr
{di counts the coins of denomination ci}
```

2.15 美国硬币系统的最优性

引理

若用 25、10、5、1 分硬币以最少硬币数表示某个正整数金额，则非 25 分硬币部分至多包含 2 枚 10 分、至多 1 枚 5 分、至多 4 枚 1 分，且不能同时包含 2 枚 10 分和 1 枚 5 分。因此非 25 分硬币的总金额不超过 24 分。

证明思路：反证。

- 若有 3 枚 10 分，可替换为 1 枚 25 分和 1 枚 5 分，硬币数更少。

- 若有 2 枚 5 分，可替换为 1 枚 10 分。
- 若有 5 枚 1 分，可替换为 1 枚 5 分。
- 若有 2 枚 10 分和 1 枚 5 分，可替换为 1 枚 25 分。

定理：对美国硬币系统，贪心找零算法总能产生硬币数最少的找零方案。

证明思路：若存在某个金额 n 的最优方案使用的硬币数少于贪心方案，设贪心方案使用 q 枚 25 分硬币，最优方案使用 q' 枚。若 $q' < q$，则最优方案的非 25 分硬币部分金额至少为 25 分，违背引理。因此最优方案必须使用同样数量的 25 分硬币。对剩余金额继续比较 10 分、5 分和 1 分硬币，可得两种方案硬币数相同，矛盾。

2.16 面值系统会影响贪心正确性

贪心找零是否最优依赖于硬币面值集合。美国硬币系统加入 50 分和 100 分后仍保持最优；但若只允许 25、10、1 分硬币，贪心算法不再总是最优。

例如 31 分：

- 贪心：$25+1+1+1+1+1+1$，共 7 枚；
- 最优：$10+10+10+1$，共 4 枚。

2.17 贪心调度

问题：给定若干报告的开始与结束时间，在一个报告厅中安排尽可能多的报告。假设：

- 报告一旦开始就持续到结束；
- 同一时刻不能有两个报告进行；
- 一个报告可以在另一个报告结束的同一时刻开始；
- 已经选择若干报告后，不能再加入与其中任一报告重叠的报告。

可能的局部选择包括：选择开始最早的兼容报告、选择持续时间最短的兼容报告、选择结束最早的兼容报告。前两种策略一般不正确；正确的贪心策略是**每一步选择结束时间最早且与已选报告兼容的报告**。

反例：选择最短报告不一定最优

报告	开始	结束	持续时间
1	8:00	9:45	105 分钟
2	9:00	10:00	60 分钟
3	9:45	11:00	75 分钟

若先选最短报告 2，就不能再选报告 1 或 3，只能安排 1 场；但选择报告 1 和报告 3 可以安排 2 场。

```
procedure schedule(talks with start times s1, ..., sn and end times e1, ..., en)
    sort talks by finish time so that e1 <= e2 <= ... <= en
    S := empty set
    for j := 1 to n
        if talk j is compatible with S then
            S := S union {talk j}
    return S
```

课件说明该算法的正确性将在第 5 章用归纳法证明。补充地看，它的直觉是：越早结束，留给后续报告的空间越大；通过交换论证也可以证明该选择不会损失最优性。

2.18 停机问题（拓展）

停机问题 (halting problem) 问：是否存在一个通用过程，能够接收任意程序 P 和输入 I，并判断 P 在输入 I 上最终会停止还是永远运行?

假设存在过程 $H(P, I)$：

- 若 P 在输入 I 上会停止，$H(P, I)$ 输出 `halt`；
- 否则，$H(P, I)$ 输出 `loops forever`。

因为程序本身也可以看作字符串，可以构造过程 $K(P)$：

- 若 $H(P, P)$ 输出 `loops forever`，则 $K(P)$ 停止；
- 若 $H(P, P)$ 输出 `halt`，则 $K(P)$ 进入无限循环并不断打印 `ha`。

现在考虑 $K(K)$：

- 若 $H(K, K)$ 判断 $K(K)$ 会永远循环，则按 K 的定义，$K(K)$ 应停止，矛盾。
- 若 $H(K, K)$ 判断 $K(K)$ 会停止，则按 K 的定义，$K(K)$ 应永远循环，矛盾。

因此，不存在能够判定任意程序是否停机的通用算法，停机问题是不可解的。

3 函数增长与渐近效率

3.1 选择算法时考虑的因素

选择算法时不仅看速度，还要综合考虑：

- **简单性 (Simplicity)**：易于实现且正确性明显。
- **清晰性 (Clarity)**：易读、易维护。
- **可扩展性 (Extensibility)**：容易扩展到新任务。
- **可复用性 (Reusability)**：可适配不同任务。

- **效率 (Efficiency)**：合理使用时间与空间。

3.2 算法分析的目标

分析 (analysis) 根据输入规模估计算法运行时间。它的作用包括：

- 描述运行时间的相对增长率，而不是某台机器上的精确执行秒数；
- 允许在没有具体实现的情况下比较算法；
- 不依赖特定硬件；
- 在给定输入规模时，可描述操作次数的精确表达式或渐近估计。

输入规模 (input size) 取决于问题本身。排序算法通常用待排序元素个数 n；图算法常同时使用顶点数 $|V|$ 和边数 $|E|$。

3.3 渐近效率

渐近效率 (asymptotic efficiency) 关心的是输入规模无界增长时，算法运行时间或空间需求如何增长。渐近记号把满足某些增长条件的函数归为一类，从而刻画算法复杂度。

3.4 Big-O 记号

3.5 定义与直观含义

Big-O 定义

设 f, g 是从 $\mathbb{N}$（或 $\mathbb{R}$）到 $\mathbb{R}$ 的函数。若存在正常数 C 和 k，使得对所有 $x \geq k$ 都有

$$|f(x)| \leq C|g(x)|,$$

则称 $f(x)$ 是 $O(g(x))$，记作 $f(x) = O(g(x))$。

直观上，$g(x)$ 给出了 $f(x)$ 的一个**渐近上界**，常数倍差异和有限个小规模输入不会影响 Big-O 判断。

3.6 Big-O 的注意事项

1. “$f(x)$ is $O(g(x))$” 通常写作 $f(x) = O(g(x))$。
2. 满足定义的 C, k 从不唯一；一旦存在一组，就存在无穷多组。
3. 若 $f(x) = O(g(x))$，且 $h(x)$ 在充分大的 x 上至少和 $g(x)$ 一样大，则 $f(x) = O(h(x))$。
4. 对数底数在 Big-O 中通常不重要，因为不同底数只相差常数倍。

3.7 Big-O 例题

例 1：证明 $x^2+2x+1=O(x^2)$

当 $x\geq 1$ 时，

$$x^2+2x+1\leq x^2+2x^2+x^2=4x^2.$$

因此取 $C=4,k=1$ 即可。也可以在 $x>2$ 时取 $C=3$，因为

$$x^2+2x+1\leq x^2+x^2+x^2=3x^2.$$

同时，$x^2=O(x^2+2x+1)$，所以这两个函数具有相同阶。

例 2：证明 $7x^2=O(x^3)$，并判断 $x^3=O(7x^2)$ 是否成立

当 $x>7$ 时，$7x^2<x^3$，因此取 $C=1,k=7$ 可得 $7x^2=O(x^3)$。

反过来若 $x^3\leq C(7x^2)$，则必须有 $x\leq 7C$。不存在固定常数 C 能让该式对所有充分大的 x 成立，因此 x^3 不是 $O(7x^2)$。

3.8 多项式的 Big-O

多项式上界

若

$$f(x)=a_nx^n+a_{n-1}x^{n-1}+\cdots+a_1x+a_0,$$

其中 $a_0,a_1,\ldots,a_n$ 为实数，则 $f(x)=O(x^n)$。

证明：当 $x>1$ 时，

$$\begin{aligned}|f(x)|&\leq |a_n|x^n+|a_{n-1}|x^{n-1}+\cdots+|a_1|x+|a_0|\\&=x^n\left(|a_n|+\frac{|a_{n-1}|}{x}+\cdots+\frac{|a_1|}{x^{n-1}}+\frac{|a_0|}{x^n}\right)\\&\leq x^n(|a_n|+|a_{n-1}|+\cdots+|a_1|+|a_0|).\end{aligned}$$

取 $C=|a_n|+|a_{n-1}|+\cdots+|a_1|+|a_0|$ 即可。

3.9 常见估计

前 n 个正整数之和

$$1+2+\cdots+n\leq n+n+\cdots+n=n^2,$$

因此

$$1+2+\cdots+n=O(n^2).$$

补充：精确公式为 $\frac{n(n+1)}{2}$，所以更紧确地说是 $\Theta(n^2)$。

阶乘函数

$$n! = 1 \cdot 2 \cdots n \le n \cdot n \cdots n = n^n,$$

因此 $n! = O(n^n)$。

$\log(n!)$

由 $n! \le n^n$ 得

$$\log(n!) \le \log(n^n) = n \log n,$$

因此 $\log(n!) = O(n \log n)$。

3.10 常见复杂度函数

复杂度	名称
$O(1)$	常数复杂度 (constant)
$O(\log n)$	对数复杂度 (logarithmic)
$O(n)$	线性复杂度 (linear)
$O(n \log n)$	$n \log n$ 复杂度
$O(n^b)$	多项式复杂度 (polynomial)
$O(b^n),\ b > 1$	指数复杂度 (exponential)
$O(n!)$	阶乘复杂度 (factorial)

常见增长速度从慢到快大致为：

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec b^n \prec n!.$$

3.11 函数组合的增长

加法

若 $f_1(x) = O(g_1(x))$ 且 $f_2(x) = O(g_2(x))$，则

$$(f_1 + f_2)(x) = O(\max(g_1(x), g_2(x))).$$

例：求 $n^2 + \log(n!)$ 的复杂度。

$$n^2 = O(n^2), \qquad \log(n!) = O(n \log n).$$

因为 n^2 比 $n \log n$ 增长更快，所以

$$n^2 + \log(n!) = O(n^2).$$

乘法

若 $f_1(x) = O(g_1(x))$ 且 $f_2(x) = O(g_2(x))$，则

$$(f_1 f_2)(x) = O(g_1(x) g_2(x)).$$

例：求 $3n \log(n!)$ 的复杂度。

$$3n = O(n), \qquad \log(n!) = O(n \log n),$$

因此

$$3n \log(n!) = O(n^2 \log n).$$

3.12 Big-Omega 与 Big-Theta

3.13 Big-Omega

Big-O 只提供渐近上界；若需要渐近下界，就使用 Big-Omega。

Big-Omega 定义

设 f, g 是从 $\mathbb{N}$（或 $\mathbb{R}$）到 $\mathbb{R}$ 的函数。若存在正常数 C 和 k，使得对所有 $x > k$ 都有

$$|f(x)| \geq C|g(x)|,$$

则称 $f(x)$ 是 $\Omega(g(x))$，记作 $f(x) = \Omega(g(x))$。

例：证明 $8x^3 + 5x^2 + 7 = \Omega(x^3)$。当 $x \geq 0$ 时，

$$8x^3 + 5x^2 + 7 \geq 8x^3.$$

取 $C = 8, k = 0$，可得结论。

3.14 Big-Theta

Big-Theta 同时给出渐近上界和下界，表示两个函数具有相同增长阶。

Big-Theta 定义

设 f, g 是从 $\mathbb{N}$（或 $\mathbb{R}$）到 $\mathbb{R}$ 的函数。若 $f(x) = O(g(x))$ 且 $f(x) = \Omega(g(x))$，则称 $f(x) = \Theta(g(x))$。等价地，存在正常数 C_1, C_2, k，使得对所有 $x \geq k$，

$$0 \leq C_1 g(x) \leq f(x) \leq C_2 g(x).$$

例：证明 $3x^2 + 8x\log x = \Theta(x^2)$

当 $x \geq 1$ 时，$\log x \leq x$，因此

$$3x^2 + 8x\log x \leq 3x^2 + 8x^2 = 11x^2,$$

所以 $f(x) = O(x^2)$。同时

$$3x^2 + 8x\log x \geq 3x^2,$$

所以 $f(x) = \Omega(x^2)$。取 $C_1 = 3, C_2 = 11, k = 1$，得

$$3x^2 + 8x\log x = \Theta(x^2).$$

4 算法复杂度

4.1 复杂度的含义

算法或程序的复杂度表示执行成本，包括运行时间、内存需求等；它不表示编写程序本身的成本，例如代码行数、开发时间等。

- **时间复杂度 (time complexity)**：求解规模为 n 的问题所需操作次数的近似增长。
- **空间复杂度 (space complexity)**：求解规模为 n 的问题所需内存量的近似增长，通常与所使用的数据结构密切相关。

4.2 三类分析

分析类型	含义
最坏情形分析 (Worst-case)	给定输入规模下操作次数的最大值；它给出对所有输入都成立的保证。
最好情形分析 (Best-case)	给定输入规模下操作次数的最小值；通常实际参考价值较低。
平均情形分析 (Average-case)	在假设输入服从某种概率分布时的平均操作次数；往往较复杂。

4.3 基本语句复杂度规则

- 简单语句，如变量初始化，复杂度为 $O(1)$。
- 循环复杂度为 $O(g(n)f(n))$，其中 $g(n)$ 是循环次数上界，$f(n)$ 是循环体复杂度上界。
- 条件语句复杂度为 $O(\max(f(n), g(n)))$，其中 $f(n)$、$g(n)$ 分别是 then 分支和 else 分支的复杂度上界。

- 若一个语句块由复杂度为 $f_1(n), f_2(n), \ldots, f_k(n)$ 的语句组成，则整体复杂度为

$$O(f_1(n) + f_2(n) + \cdots + f_k(n)).$$

4.4 求最大元素的复杂度

对求最大元素算法，若按比较次数计：

- 数据比较 `max < ai` 进行 $n-1$ 次；
- 循环条件检查进行 n 次；
- 总比较次数可记为 $2n-1$。

因此算法时间复杂度为 $\Theta(n)$。若采用不同的计数约定，精确常数可能不同，但渐近阶不变。

4.5 线性搜索复杂度

最坏情形

在线性搜索中，每一步需要比较 $i \le n$ 和 $x \ne a_i$。若目标元素在最后或不存在，算法需要扫描到列表末尾，比较次数与 n 成正比。课件按一种计数方式得到最坏情形比较次数为 $2n+2$，因此最坏时间复杂度为

$$\Theta(n).$$

平均情形

假设目标元素一定在列表中，且出现在每个位置的概率相同。若 $x = a_i$，比较次数为 $2i+1$。平均比较次数为

$$\frac{3 + 5 + 7 + \cdots + (2n+1)}{n} = \frac{2(1 + 2 + \cdots + n) + n}{n} = n + 2.$$

因此线性搜索的平均情形复杂度也是 $\Theta(n)$。

4.6 二分搜索复杂度

假设 $n = 2^k$，则 $k = \log n$。二分搜索每轮把搜索区间规模减半：

$$2^k, 2^{k-1}, 2^{k-2}, \ldots, 2, 1.$$

每轮至多做常数次比较，因此最多约为

$$2k + 2 = 2\log n + 2$$

次比较。故二分搜索复杂度为

$$\Theta(\log n),$$

优于线性搜索。但它依赖一个前提：输入列表必须已经有序。

4.7 冒泡排序复杂度

冒泡排序第 i 趟内层循环进行 $n-i$ 次比较，因此比较总数为

$$(n-1)+(n-2)+\cdots+2+1=\frac{n(n-1)}{2}=\frac{1}{2}n^2-\frac{1}{2}n.$$

因此最坏情形复杂度为

$$\Theta(n^2).$$

4.8 插入排序复杂度

课件按比较次数给出插入排序最坏情形下的总比较次数：

$$2+3+\cdots+n=\frac{n(n-1)}{2}-1.$$

因此复杂度为

$$\Theta(n^2).$$

补充：插入排序在几乎有序的输入上通常表现较好；在已经有序时可达到线性级别，但最坏情况仍为二次级别。

4.9 矩阵乘法复杂度

用标准算法相乘两个 $n\times n$ 整数矩阵。乘积矩阵有 n^2 个元素，每个元素需要：

- n 次乘法；
- $n-1$ 次加法。

因此总共需要 n^3 次乘法和 $n^2(n-1)$ 次加法，复杂度为

$$O(n^3).$$

4.10 布尔矩阵乘积复杂度

对两个 $n\times n$ 零一矩阵 A 与 B，其布尔乘积 $A\odot B$ 的每个元素需要 n 次 OR 和 n 次 AND，共 $2n$ 次位操作。因为共有 n^2 个元素，总操作次数为

$$2n^3,$$

所以复杂度为

$$O(n^3).$$

4.11 复杂度类别与 P/NP（拓展）

4.12 常见术语

- **可处理问题 (tractable problem)**：存在多项式时间算法可解的问题，属于类 P。
- **难处理问题 (intractable problem)**：不存在多项式时间算法的问题。
- **不可解问题 (unsolvable problem)**：不存在任何算法可解的问题，例如停机问题。
- **类 NP**：给定一个候选解后，能够在多项式时间内验证其正确性的问题类。
- **NP 完全类 (NP-complete)**：若该类中任意一个问题有多项式时间算法，则 NP 中所有问题都有多项式时间算法。

4.13 P 与 NP 问题

P versus NP 问题询问：是否所有能在多项式时间内验证解的问题，也都能在多项式时间内找到解？也就是是否 $P = NP$。

课件中的典型例子：

- **旅行商问题 (TSP) 的判定版**：给定地图和预算 B，是否存在一条经过所有城市且总花费 $\leq B$ 的路径？求解很难，但验证一条给定路径是否满足预算很容易。
- **布尔可满足性问题 (SAT)**：给定逻辑公式，是否存在一组变量赋值使公式为真？求解困难，但验证某组赋值是否有效很简单。SAT 是一个 NP 完全问题。
- **数独**：给定填好的候选解，可以快速检查是否合法；寻找解则可能困难。

通常认为 $P \neq NP$, 因为至今没有人找到任一 NP 完全问题的多项式时间算法。Clay Mathematics Institute 为该问题的解决提供了 $\$1,000,000$ 奖金。

补充说明

课件提到大整数分解与密码学安全。严格地说，大整数分解属于 NP，但尚未证明它是 NP 完全或通常意义下的 NP 难问题；现有 RSA 等密码体系的安全性依赖于目前没有已知的高效经典算法来分解大整数。若 $P = NP$ 且相应算法可实际运行，密码学、优化、人工智能与自动定理证明都会受到深远影响。

5 全章总结

1. 算法是解决问题的有限、精确、有效且一般化的步骤集合。
2. 搜索、排序、字符串匹配和优化问题是算法分析中最基础的例子。
3. 贪心算法每步做局部最优选择，但必须证明其全局最优性；美国硬币找零和最早结束时间调度是经典成功案例。

4. Big-O、Big-Omega、Big-Theta 分别描述渐近上界、渐近下界和精确增长阶。
5. 算法复杂度关注运行成本，而不是代码开发成本；最坏情形给保证，平均情形依赖输入分布。
6. 线性搜索为 $\Theta(n)$，二分搜索为 $\Theta(\log n)$，冒泡排序和插入排序最坏为 $\Theta(n^2)$，标准矩阵乘法为 $O(n^3)$。
7. 停机问题说明并非所有问题都有算法可解；P 与 NP 问题则刻画“能快速验证”与“能快速求解”之间的深刻差异。

5.1 课件习题

- 第 3.1 节：P213，6、20、47、60。
- 第 3.2 节：P228，19、36–37、45、53、54、71。
- 第 3.3 节：P241，12、37。

“算法分析关心的不是今天这台机器跑了多久，而是问题变大时，代价会如何增长。”